

WB Margin Analyzer — Итоговое ТЗ v2.0

Идея продукта

Веб-инструмент для анализа маржинальности товаров на Wildberries. Загружаешь два Excel-файла → получаешь реальную чистую маржу по каждому товару, интерактивный симулятор "что если", AI-интерпретацию от Claude и цветовой Excel-отчёт. Инструмент принятия решений, а не просто генератор отчётов.

Стек

Слой	Технологии
Backend	Python 3.11 · FastAPI · pydantic-settings
Аналитика	pandas · openpyxl
База данных	PostgreSQL · SQLAlchemy (async) · Alembic
AI	Claude API (claude-sonnet-4-6)
Auth	JWT (python-jose · passlib)
Frontend	React 18 · Vite · TailwindCSS
Инфраструктура	Docker Compose
Тесты	pytest · pytest-asyncio

Входные данные

Файл 1 — Закупки (purchases.xlsx)

Артикул	Название	Закупочная цена	Категория WB	Вес (кг)
WB-001	Крем для лица SPF50	320	Косметика	0.2

Файл 2 — Продажи (sales.xlsx)

Артикул	Цена продажи	Продано штук	Возвраты штук
WB-001	890	145	12

Логика расчёта маржи

Комиссии WB по категориям:

- Косметика: 15% · Электроника: 10% · Игрушки: 12% · Одежда: 23% · Прочее: 18%

Формулы (скорректированные):

Возврат_коэф	= возвраты / продано
Выручка_чистая	= цена_продажи × продано × (1 – возврат_коэф)
Комиссия_сумма	= выручка_чистая × комиссия_%
Логистика_туда	= вес × 80 × продано
Логистика_обратно	= вес × 80 × возвраты × 1.5 ← WB берёт коэф за возврат
Логистика_итого	= логистика_туда + логистика_обратно
Затраты_закупка	= закупочная_цена × продано
Прибыль	= выручка_чистая – комиссия_сумма – логистика_итого – затраты_закупка
Маржа_%	= (прибыль / выручка_чистая) × 100

Пользовательский поток

Шаг 0 — Авторизация

Регистрация / логин. JWT токен в localStorage. Все данные привязаны к пользователю.

Шаг 1 — Загрузка и валидация файлов

- Две drag-and-drop зоны
- После загрузки: **preview до расчёта** — показываем распознанные колонки, кол-во строк, предупреждения

✓ Обнаружено 20 товаров
 ⚠ Колонка "Вес" отсутствует – логистика рассчитана по среднему (0.5 кг)
 Preview: WB-001 | Крем для лица | 320₽ | Косметика

- Кнопка "Запустить анализ" только после подтверждения preview

Шаг 2 — Асинхронная обработка

- `POST /api/v1/analyses` → `202 Accepted` + `{ analysis_id, status: "processing" }`
- Фронт делает polling каждые 2 секунды
- Прогресс-бар с текстом: "Читаем файлы → Считаем маржу → Запрашиваем AI..."

Шаг 3 — Дашборд результатов

Сводные метрики сверху:

- Общая выручка · Общая прибыль · Средняя маржа · Кол-во товаров в каждой зоне

Таблица с цветовой маркировкой:

-  Зелёный — маржа > 25%
-  Жёлтый — маржа 10–25%
-  Красный — маржа < 10% или убыток

Топ-блоки:

- Топ-3 прибыльных · Топ-3 убыточных

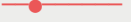
Шаг 4 — What If симулятор

Рядом с каждым товаром — раскрывающаяся панель:

WB-002 Маржа: 7% 

[Цена продажи: 2490] 

3200 → Маржа: 19% 

[Закупочная: 890] 

700 → Маржа: 24% 

Применить изменения?

Пересчёт мгновенный, на клиенте (JS-реализация тех же формул). Сохранять как "сценарий" не нужно — это живой интерактив.

Шаг 5 — AI-интерпретация (Claude API)

Блок с текстовым анализом после расчёта. Структурированный промпт:

1. Диагноз: что плохо и почему
2. Рекомендации: конкретные действия (цена, закупка, категория)
3. Риски: на что обратить внимание

Мини-чат под интерпретацией:

```
Пользователь: "А что если снизить закупку WB-002 до 700₽?"
Claude: "При закупке 700₽ и текущей цене маржа составит 14%..."
```

Контекст = результаты анализа + история диалога (последние 5 сообщений).

Шаг 6 — История и сравнение

Список прошлых анализов. Кнопка "Сравнить" — появляется колонка Δ Маржа с ↑↓.

Шаг 7 — Экспорт отчёта

Excel-файл с цветовыми ячейками (openpyxl conditional formatting) + лист с AI-выводами.

База данных

```
users          - id, email, hashed_password, created_at
analysis_sessions - id, user_id, status, created_at, files_meta (JSON)
products       - id, session_id, артикул, название, все расчётные поля
reports        - id, session_id, ai_interpretation, chat_history (JSON)
```

Структура проекта

```
wb-margin-analyzer/
├── backend/
│   ├── api/v1/
│   │   ├── auth.py
│   │   ├── uploads.py
│   │   ├── analyses.py
│   │   └── reports.py
│   ├── services/
│   │   ├── analyzer.py      ← pandas логика
│   │   ├── ai_service.py    ← Claude API
│   │   └── excel_service.py ← openpyxl экспорт
│   ├── models/              ← SQLAlchemy
│   ├── schemas/              ← Pydantic request/response
│   ├── core/
│   │   ├── config.py        ← pydantic-settings + .env
│   │   ├── security.py      ← JWT
│   │   └── exceptions.py
│   └── tests/
│       ├── test_analyzer.py ← unit тесты формул
│       └── test_api.py
└── frontend/src/
```

```
| | components/
| | | Upload/
| | | Dashboard/
| | | WhatIfSimulator/
| | | AnalysisChat/
| | hooks/
| | | usePolling.js ← polling статуса анализа
| | | utils/
| | | | margin.js ← те же формулы на JS
| | scripts/
| | | generate_test_data.py
| | alembic/
| | docker-compose.yml
| | .env.example
| | README.md
```

Порядок разработки

1. Генератор тестовых данных (20 товаров, разная маржинальность)
 2. PostgreSQL модели + Alembic миграции
 3. JWT авторизация
 4. Validation layer + preview эндпоинт
 5. pandas-логика расчёта маржи + pytest
 6. FastAPI эндпоинты с async обработкой
 7. Claude API интерпретация + мини-чат
 8. React фронтенд (Upload → Dashboard → WhatIf → Chat)
 9. Экспорт Excel-отчёта
 10. Docker Compose + .env.example
 11. README с архитектурной схемой
-

Питч для резюме

WB Margin Analyzer — AI-powered decision tool for marketplace sellers.
FastAPI · pandas · PostgreSQL · Claude API · React · Docker

Upload cost + sales Excel → get margin analysis with color coding, interactive
"What If" price simulator, and Claude AI interpretation with follow-up Q&A.

| Async processing, JWT auth, pytest coverage.